No CUDA, No Problem: Lattice QCD on AWS Trainium and Inferentia via PyTorch–XLA

João Galego[1]

[1] *Independent Researcher*

AI Research Assistant: Claude Opus 4.7 (Anthropic)

May 2, 2026

Abstract

We present LQCD-Neuron, a Python library that runs Wilson- and Clover-Wilson Lattice QCD on AWS Trainium (Trn1) and Inferentia (Inf2) accelerators. The hardware exposes no general-purpose compute API (no CUDA, HIP, or OpenCL); the only programming model is to compile a PyTorch / XLA graph ahead of time with `neuronx-cc` and execute the resulting NeuronCore Executable File Format (`.neff`) binary. We re-express every hot-path lattice kernel-the Wilson Dslash, the clover term, BLAS-1 reductions, and CG/BiCGStab solvers-as a pure `torch.nn.Module`, and rely on three compile-time transformations (gauge baking, fused 12×12 spin–colour matvecs, multi-RHS batching) plus a host-side data-parallel shard across all on-chip NeuronCores. On a two-core Inf2 (`inf2.xlarge`) in BF16 we obtain up to 13.0× throughput over a tuned FP32 CPU baseline for the Wilson Dslash on $V = 4{\times}4{\times}4{\times}4$ (B=8 RHS per core), and establish that throughput reaches CPU parity at $V = 16{\times}8{\times}8{\times}8$ and falls below beyond that, all with no custom kernel code. We emphasise that these speedups are achieved on sub-production volumes; sustained throughput remains $\leq 7\,\mathrm{GFLOP/s}$, well below both the NeuronCore-v2 BF16 peak and published QUDA numbers for Wilson Dslash on contemporary GPUs. The work is therefore best read as a feasibility / lessons-learned study of the AoT-compiled, static-graph programming model rather than a competitive implementation. Source code is available at github.com/JGalego/lqcd-neuron under the Apache 2.0 license. All calculations, numerical analysis, and manuscript preparation were performed by Claude, an AI assistant developed by Anthropic, working under physicist supervision.

1 Introduction

The cost of a Lattice QCD simulation on heterogeneous hardware is dominated by repeated applications of the discretised Dirac operator D to a quark spinor field ψ. In the Wilson formulation [1],

$$(D_W\psi)_x = (4+m)\,\psi_x - \tfrac{1}{2}\sum_{\mu=0}^{3}\Big[(I-\gamma_\mu)\,U(x,\mu)\,\psi_{x+\hat{\mu}} + (I+\gamma_\mu)\,U^\dagger(x-\hat{\mu},\mu)\,\psi_{x-\hat{\mu}}\Big], \qquad (1)$$

with gauge links $U(x,\mu)\in\mathrm{SU}(3)$ and Dirac matrices γ_μ acting on the four-component spinor. We use the unhopped (mass-form) normalisation; the equivalent κ-form, $\kappa = 1/(2(m+4))$, is recovered by rescaling ψ. Anti-periodic temporal boundary conditions are imposed by multiplying the temporal gauge links on the boundary timeslice by -1, after which all shifts can be implemented as periodic `torch.rolls`. performs $\mathcal{O}(V)$ tiny dense 3×3 and 4×4 contractions,

and a Krylov inversion invokes it tens of thousands of times. On NVIDIA GPUs the standard tool is QUDA [2, 3]: hand-written CUDA kernels, dynamic shapes, runtime auto-tuning, and a multigrid preconditioner [4]. We ask whether comparable performance is reachable on AWS-designed ML accelerators - Trainium (Trn1) and Inferentia (Inf2) - which expose a fundamentally different programming model.

Each NeuronCore-v2 is a purpose-built matrix engine. It offers no general-purpose compute API, no user-controlled shared memory, and no runtime kernel launch. Code reaches the hardware along a single path: a PyTorch (or JAX) computation graph is lowered ahead of time to XLA HLO [5], the `neuronx-cc` compiler emits a `.neff` binary, and the Neuron runtime executes it on the NeuronCores. Tensor shapes must be static at compile time, and the auto-tuning and runtime kernel re-launch on which QUDA relies are simply not available. Trainium has been shown to scale deep-learning training [6], but to our knowledge has not previously been targeted for first-principles Lattice QCD.

The central question of this work is therefore: *is the AoT-compiled, static-graph, matrix-engine programming model expressive enough for production Lattice QCD primitives, and competitive in throughput once the graph is shaped to fit the hardware?* We give an affirmative answer for the Wilson and Clover-Wilson Dslash and the iterative solvers built on them, with four composable graph-level optimisations that bring throughput from below CPU at modest volumes to up to 13.0× above it on a two-core Inf2 configuration.

2 Design and architecture

Pure-tensor kernels. Every primitive in LQCD-Neuron is a standard `torch.nn.Module` operating on dense complex tensors with a $(T, Z, Y, X, \dots)$ storage order. Gauge links live in $U \in \mathbb{C}^{T\times Z\times Y\times X\times 4\times N_c\times N_c}$ and spinors in $\psi \in \mathbb{C}^{T\times Z\times Y\times X\times N_s\times N_c}$, with $N_c = 3,\ N_s = 4$. Lattice shifts $\psi_{x\pm\hat{\mu}}$ are realised by `torch.roll` with the appropriate axis index, which the Neuron compiler lowers to in-place data movement on chip. Since `neuronx-cc` forbids data-dependent control flow, the Python loop over the four directions in Eq. (1) is statically unrolled at trace time, producing a pure DAG that the compiler can schedule.

Gamma matrices. The Euclidean Dirac matrices are stored in the DeGrand–Rossi basis [7] and registered as non-trainable `nn.Module` buffers, so they migrate with the module to NeuronCore memory under `torch_neuronx.trace` and remain visible to the tracer.

Host loop / device kernel split. Iterative solvers need a convergence test on a scalar reduction at every iteration, but `neuronx-cc` only supports static graphs. We adopt the standard deep-learning split: the Krylov loop runs on the host (CPU) and only the matvec $D\psi$ is compiled to NeuronCore. For Conjugate Gradient [8] (and analogously BiCGStab [**?**]), schematically:

```
for k in range(maxiter):
    Ap = D_neuron(p, U) # NeuronCore
    alpha = rho / inner(p, Ap) # host
    x += alpha * p
    r -= alpha * Ap
    if norm(r) < tol: break
```

The host overhead per iteration is $\mathcal{O}(V)$ and is dwarfed by the matvec for any non-trivial lattice; we verified this empirically against an all-on-host reference.

CPU fallback. Because every kernel is a plain PyTorch module, the same code runs identically on a laptop without Neuron hardware. The full unit-test suite executes on CPU, allowing development and CI without an Inf2 instance.

3 Compile-time optimisations

A naive trace of Eq. (1) is dominated by overheads that are absent on a CPU baseline: (i) the gauge field, $\sim 3\times$ the spinor in size, crosses PCIe on every call; (ii) the per-direction kernel is two tiny einsums (a 3×3 colour multiply and a 4×4 spin multiply), neither of which saturates the NeuronCore tensor engine; (iii) eight `torch.roll`s per call (four for ψ, four for $U^\dagger(x-\hat{\mu})$) are pure data movement; and (iv) every NeuronCore call pays a fixed dispatch cost ($\sim 1\,\mathrm{ms}$), independent of problem size. We address these in four composable steps.

3.1 Gauge baking

When a gauge configuration U is supplied at compile time, we register it as an `nn.Module` buffer of the traced module rather than as a graph input. After `torch_neuronx.trace`, the resulting `.neff` contains U in NeuronCore-resident memory. At inference time, only ψ crosses PCIe; the wrapper accepts a U argument for API compatibility but ignores it. The trade-off is that the `.neff` is specific to a particular U, so it is not reused across different gauge configurations - the right choice for a single inversion or a chain of solves on one configuration, the wrong choice for HMC.

3.2 Fused spin–colour kernels

With U baked in, the per-site, per-direction operator

$$K^{\mathrm{fwd}}_{\mu,x} = (I-\gamma_\mu)\otimes U(x,\mu),\ K^{\mathrm{bwd}}_{\mu,x} = (I+\gamma_\mu)\otimes U^\dagger(x-\hat{\mu},\mu) \tag{2}$$

can be precomputed once at trace time. We materialise $K^{\mathrm{fwd/bwd}}_{\mu,x} \in \mathbb{C}^{N_sN_c\times N_sN_c}$ as four buffers of shape $(4,T,Z,Y,X,12,12)$ stored as real/imaginary float pairs. Each direction-side then collapses to one 12×12 matvec plus a single roll of ψ:

```
psi_shifted = torch.roll(psi, sign, dim=mu)
result -= 0.5 * (K[mu] @ psi_shifted)
```

The 12×12 shape is markedly better suited to the NeuronCore matrix multiplication unit (MXU) than the original 3×3 / 4×4 einsums. The four backward rolls are absorbed into K^{bwd}, halving the lattice-shift count. The fused kernel agrees with the reference `forward()` to a relative error $\lesssim 10^{-7}$ in FP32, and continues to satisfy the existing γ_5-Hermiticity, adjoint, and normal-operator positivity tests.

SRAM budget guard. The eight $K^{\mathrm{fwd/bwd}}$ buffers have per-lattice footprint $8V\cdot144\cdot8$ bytes (complex entries stored as a pair of float32 reals). This already exceeds the 14.4 MiB NeuronCore on-chip SRAM at $V = 16\times8\times8\times8$ (72 MiB) and reaches 576 MiB at $V = 16\times16\times16\times16$. The compiler therefore checks the fused footprint against a configurable SRAM threshold before tracing; if the budget is exceeded it issues a warning and falls back to the unfused baked-gauge path, preserving correctness at reduced MXU efficiency. The threshold can be overridden via `NeuronCompiler(sram_threshold_bytes=N)` or suppressed with `fused=False`.

3.3 Multi-right-hand-side batching

Production multi-source propagator inverters already solve $Dx = b_i$ for $N_s\cdot N_c = 12$ right-hand sides per source, one per spin $\times$ colour combination. We expose this directly, `compile_dslash_batched(`D, shape, `batch_size=`B, `gauge_field=`U`)`, which traces the fused operator on $\psi \in \mathbb{C}^{B\times T\times Z\times Y\times X\times N_s\times N_c}$. The baked K buffers carry a singleton batch axis that broadcasts across all B RHS, and the lattice rolls use negative-dim indexing so the leading batch axis is unaffected. Each direction-side becomes an $N_b\times12\times12$ matmul - a much better

Table 1: Wilson Dslash sustained throughput on an `inf2.8xlarge` (BF16, gauge baked) versus a tuned CPU baseline (FP32), in Dslash applications per second over 50 iterations after a 5-iteration warm-up. Neuron = 1 NeuronCore, 1 RHS; Batched = 1 NeuronCore, B=8 RHS; Multicore = 2 NeuronCores, B=8 RHS each. Speedup is the Multicore column relative to CPU; entries at or below 1× indicate the optimised Neuron path has not yet overcome the dispatch and bandwidth bottleneck at that volume. GFLOP/s and GB/s are derived from the Multicore column (1320 flop/site [3]; streaming spinor in+out, BF16; gauge baked, no PCIe traffic for U). For context, NeuronCore-v2 BF16 peak is ~ 95 TFLOP/s per core and HBM bandwidth is ~ 400 GB/s per chip, so the sustained figures correspond to $\lesssim 10^{-4}$ of compute peak and $\lesssim 10^{-3}$ of memory peak. † Fused kernels exceed the 14.4 MiB NeuronCore SRAM budget at this volume; the compiler automatically falls back to the unfused baked-gauge path.

Lattice	CPU	Neuron	Batched	Multicore	Speedup	GFLOP/s	GB/s
4×4×4×4	804.6	1358.0	8521.5	**10431.3**	13.0×	3.52	0.26
8×4×4×4	769.5	785.1	4786.3	**7519.1**	9.8×	5.08	0.37
8×8×4×4	518.1	370.1	2511.1	**4229.5**	8.2×	5.72	0.42
8×8×8×4	384.0	207.6	1390.1	**2474.2**	6.4×	6.69	0.49
V = 16×8×8×8	229.2	37.2	253.8	220.1	0.96×†	2.38	0.17
16×16×16×16	59.6	1.2	24.4	15.8	0.3×†	1.36	0.10

fit for the MXU's systolic schedule - and the fixed per-call dispatch cost is amortised over B problems.

3.4 Multi-core data-parallel sharding

A single NeuronCore-v2 is one of N_{core} cores per accelerator chip; `inf2.xlarge` exposes 2, `trn1.32xlarge` 32. We replicate the compiled `.neff` across all detected cores via `torch_neuronx.DataParallel` and split a global batch of $N_{\text{core}} \cdot B_{\text{core}}$ RHS along dim 0:

```
D_mc = compiler.compile_dslash_multicore(
    D, shape, gauge_field=U,
    num_cores=N, per_core_batch_size=B,
)
out = D_mc(psi_global) # B_g = N * B
```

This composes with the previous three optimisations rather than replacing them: gauge baking and fused kernels apply per core, the intra-core multi-RHS batch fills each core's tensor engine, and the data-parallel shard multiplies by N_{core}.

4 Results

Table 1 reports the throughput of the optimised LQCD-Neuron Wilson Dslash on an `inf2.8xlarge`, compared against a CPU baseline running the same reference `nn.Module` in FP32. Three observations are worth noting.

First, the unoptimised single-RHS Neuron column (column 3) is only faster than CPU at the smallest volume; by $V = 8{\times}8{\times}8{\times}4$ it is slower by nearly 2×, and at $V = 16{\times}16{\times}16{\times}16$ by more than 8×. This confirms that the naive Dslash graph is dispatch- and shape-limited: the work per call is too small to amortise the ~ 1 ms NeuronCore launch cost, and the underlying 3×3 / 4×4 einsums leave the MXU idle. Second, multi-RHS batching closes the gap and overtakes CPU at all volumes up to $V = 8{\times}8{\times}8{\times}4$, but reaches only CPU parity at $V = 16{\times}8{\times}8{\times}8$ (1.0× with Multicore)† and falls clearly below at $V = 16{\times}16{\times}16{\times}16$ (0.3× with Multicore)†; at $V = 16{\times}8{\times}8{\times}8$ Multicore reaches 0.96× CPU, which we describe as parity to one significant figure. At both large volumes the fused 12×12 kernel buffers alone exceed the 14.4 MiB NeuronCore on-chip SRAM (72 MiB at 16×8×8×8 and 576 MiB at 16×16×16×16), so the compiler

automatically falls back to the unfused baked-gauge path, eliminating the MXU-utilisation benefit of the fused kernels. Third, the two-core `DataParallel` (Multicore) configuration delivers 6.4–13.0× over CPU from $V = 4^4$ through $V = 8\times8\times8\times4$. The breakeven volume - beyond which the CPU baseline is faster even with all optimisations - is now at $V = 16\times8\times8\times8$ (CPU parity, 1.0×); the fused-kernel SRAM limit is the primary obstacle to recovering the 2×–3× advantage that would otherwise be expected at these volumes, and bridging it (via kernel tiling or NeuronCore SRAM bucketing) is the primary motivation for the work outlined in Sec. 5.

We have additionally verified, on CPU and on a 2-core Inf2 in `DataParallel` mode, that the BiCGStab and CG host-loop solvers converge to the expected residuals on a Gaussian random ψ with anti-periodic temporal boundaries, and that the gauge-baked, fused, multi-RHS `.neff` is invariant under permutation of right-hand sides to within FP32 round-off.

5 Limitations and outlook

Functionality gap with QUDA. The current release covers only Wilson and Clover-Wilson fermions, CG and BiCGStab, periodic / anti-periodic boundaries, and a small set of gauge observables (plaquette, Wilson action, Polyakov loop, clover-definition topological charge). Staggered, twisted-mass and domain-wall formulations [9, 10, 11] are representable in the same `nn.Module` style but not yet implemented. Even-odd preconditioning and a multigrid preconditioner are the most impactful missing components; both are pure-tensor algorithms and present no fundamental obstacle to the `neuronx-cc` pipeline.

Multi-instance scaling. Within a single Inf2/Trn1 instance, communication uses NeuronLink on-chip and `torch_neuronx.DataParallel` as shown above. For multi-instance domain decomposition with halo exchange, AWS exposes EFA via `torch.distributed` with the NCCL-style XLA collective backend; integrating this with a Schwarz-style domain decomposition is the natural next step and is what is required to address physically relevant volumes ($V \gtrsim 32\times32\times32\times32$) that exceed single-chip HBM.

Static shapes. The most awkward limitation is the static-shape requirement: each tuple $(\text{class}, \text{shape}, \text{dtype}, B)$ compiles to a distinct `.neff`. For a fixed production volume this is amortised once, but for shape-sweeping workloads (auto-tuning, mixed-volume gauge ensembles) it is a real cost. `neuronx-cc` bucketing and shape-polymorphic XLA are partial mitigations.

Precision. We currently run BF16 for the matvec with FP32 accumulation in the host-side CG/BiCGStab loop, which suffices to match the existing unit-test tolerances. Mixed-precision Krylov methods with iterative refinement [2] would map cleanly onto the NeuronCore MXU's BF16 inputs / FP32 accumulators and are expected to recover full double-precision residuals at near-BF16 throughput.

6 Conclusion

We have shown that the AoT-compiled, static-graph, matrix-engine programming model exposed by AWS Trainium and Inferentia is expressive enough to host the core kernels of Wilson and Clover-Wilson Lattice QCD, and that with four straightforward graph-level optimisations - gauge baking, fused 12×12 spin–colour kernels, multi-RHS batching, and multi-core `DataParallel` sharding - a pure-Python library with no custom kernel code reaches up to 13.0× over a tuned CPU baseline on $V = 4\times4\times4\times4$ in BF16, with CPU parity at $V = 16\times8\times8\times8$ and a sustained throughput that remains far below both NeuronCore peak and published QUDA numbers. Closing that gap will require even-odd preconditioning, SU(3) reconstruction to relieve memory

pressure, and an SRAM-aware tiled fused kernel; the present paper is best read as a feasibility report establishing the programming model rather than a competitive implementation.

Acknowledgments

The author thanks the AWS Neuron SDK team for documentation and support, and the developers of PyTorch, XLA, and QUDA whose designs informed the architecture of this work.

Author Contributions

J.G. conceived and directed the project, defined the scope and target hardware, guided the AI assistant throughout development, validated the benchmark results on AWS Inferentia (Inf2) instances, and reviewed all code and prose for scientific and engineering correctness. Claude, an AI research assistant developed by Anthropic, performed the implementation work-including the `torch.nn.Module` kernel design, the four graph-level optimisations (gauge baking, fused 12×12 spin–colour kernels, multi-RHS batching, and multi-core `DataParallel` sharding), the unit-test suite, the benchmark harness, the numerical analysis, and the manuscript preparation. The work was conducted using Claude Code, Anthropic's agentic coding tool. J.G. is fully responsible for the scientific and technical content and integrity of this paper.

References

[1] K. G. Wilson, *Confinement of quarks*, Phys. Rev. D **10**, 2445 (1974).

[2] M. A. Clark, R. Babich, K. Barros, R. C. Brower, and C. Rebbi, *Solving Lattice QCD systems of equations using mixed precision solvers on GPUs*, Comput. Phys. Commun. **181**, 1517 (2010), arXiv:0911.3191.

[3] R. Babich, M. A. Clark, B. Joó, G. Shi, R. C. Brower, and S. Gottlieb, *Scaling lattice QCD beyond 100 GPUs*, SC'11 Proceedings (2011), arXiv:1109.2935.

[4] R. C. Brower, M. A. Clark, A. Strelchenko, and E. Weinberg, *Multigrid algorithm for staggered lattice fermions*, Phys. Rev. D **97**, 114513 (2018), arXiv:1801.07823.

[5] The XLA Authors, *XLA: Optimizing compiler for machine learning*, https://openxla.org/xla (2024).

[6] Amazon Web Services, *AWS Neuron SDK Documentation*, https://awsdocs-neuron.readthedocs-hosted.com/ (2024).

[7] T. A. DeGrand and D. Toussaint, eds., *From Actions to Answers*, World Scientific (1990); see also F. Berruto *et al.*, Phys. Rev. D **73**, 014503 (2006).

[8] M. R. Hestenes and E. Stiefel, *Methods of conjugate gradients for solving linear systems*, J. Res. Natl. Bur. Stand. **49**, 409 (1952).

[9] H. A. van der Vorst, *Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems*, SIAM J. Sci. Stat. Comput. **13**, 631 (1992).

[10] J. B. Kogut and L. Susskind, *Hamiltonian formulation of Wilson's lattice gauge theories*, Phys. Rev. D **11**, 395 (1975).

[11] R. Frezzotti, P. A. Grassi, S. Sint, and P. Weisz, *Lattice QCD with a chirally twisted mass term*, JHEP **08**, 058 (2001), hep-lat/0101001.

[12] D. B. Kaplan, *A method for simulating chiral fermions on the lattice*, Phys. Lett. B **288**, 342 (1992), hep-lat/9206013.